Reasoning Random Forest

Given my background and knowledge from previous AI and ML modules I've taken, I considered logistic regression for the SQLi segment of my model. However, this is not feasible given the assumptions of a logistic regression model:

- ➔ Linear boundary between classes
- ➔ Independent contribution to each feature
- ➔ Smooth decision surface

None of which are fit for SQLi detection… SQLi detection isn't linear.

- ➔ Also, my model is supposed to distinguish between three classes: 0, 1, 2; Benign, SQLi, or Emotet.

<u>The features of my dataset consist of:</u>

- Constant Value Count
- Parentheses Count
- Sentence Length
- AND Count
- OR Count
- Quote Counts
- UNION Count
- Special Character Count, etc…

These features interact with one another in order for an entry to be labeled as suspicious/malicious. For example:

- A payload with OR count of 1 and Quote Count of 1 is not necessarily malicious
- But combined with other features…: OR Count=3, AND Count=2, Constant value count=1. That would be considered malicious

Additionally, I considered a support-vector machine (SVM), since it was covered in the Year three ML module, but this model would be too slow given the size of my dataset.

Random Forest:

- ✓ Automatically handles non-linearity
- ✓ Captures feature interactions (as described above)
- ✓ Works well with tabular data
- ✓ Resistant to overfitting and bias
- ✓ Reduces variance
- ✓ Widely acceptable in cybersecurity models and literature

Before beginning, I familiarized myself with the Random Forest model by watching some videos and doing more research on what the model is and how it works in order to correctly implement it for my own model.

- What is Random Forest?
- [Random Forest Algorithm in Machine Learning - GeeksforGeeks]

How Random Forest Works

1 - Decision Trees

- It will keep splitting the dataset using feature thresholds that maximize separation using Gini impurity.
  Gini Impurity: measure of class mixing used by decision trees to select splits that maximize class purity. (aim for Gini = 0, this means it's a perfect split)
- If a tree asks 'Should I split on Constant Value Count > 0', it'll calculate Gini before and after the split, then it chooses the split that lowers impurity the most.

Addressing Problems with a Single Tree

- This is exactly what Random Forest aims to beat, one decision tree comes with limiting factors such as:
  - **X** Overfitting
  - **X** Sensitivity to noise
  - **X** Unstable, if a small change occurs in the data, the big structure is effected

2 - Random Forest Solves these Issues

- It will build many decision trees, and each is trained on
  A random subset of data
  A random subset of features at each split.
- Result: every tree sees a slightly different version of the scenario.

3 - Prediction

- Each tree makes a prediction, and the final prediction depends on the majority's vote. For example:
  Tree 1 → malicious, Tree 2 → malicious, Tree 3 → benign, Tree 4 → malicious
- That's 3 vs 1 — The final prediction is malicious.

When Random Forest is applied to the characteristics of an SQLi, each tree will have a different focus:

- Some trees may focus on quote patterns
- Some may focus on logical operators
- Others on length patterns

And what random forest will do is combine all these different perspectives to come down to one label, benign or malicious.

Building and Training My Model

Training dataset: data/updated_file.csv

Trained using 9 engineering features:

1. Sentence Length
2. AND Count
3. OR Count
4. UNION Count
5. Single Quote Count
6. Double Quote Count
7. Constant Value Count
8. Parentheses Count
9. Special Characters Total

So when I carry out external validation, any external dataset must be transformed into these 9 columns.

Performance Metrics:

- Accuracy: 0.995504
- Precision: 0.994633
- Recall: 0.993147
- F1: 0.993889
- ROC-AUC: 0.999551

Testing:

- Superviz25-SQL: SQL Injection Detection Dataset
- data/zenodo_sample_20k_features.csv — used for evaluation, setup for the expected 9 features
- data/zenodo_sample_20k.csv — raw external data
- The external dataset was balanced (10,000 benign, 10,000 malicious) to ensure class balance.

Feature Importance in Random Forest

It's the average reduction in class impurity contributed by each feature across all decision trees. Features that consistently produce strong class separation receive a higher importance score.

<u>How is importance decided?</u>

- Each decision tree will look at all available features, and choose the one that best separates the classes. (recall Gini impurity, it selects the one that reduces this value the most). It'll repeat this many times across many trees.
- A features accumulates high importance if it ticks these boxes:
    - ☐ Chosen often
    - ☐ Produces strong class separation
    - ☐ Appears near the top of the trees
- The forest averages this across all trees

In my model's case, that would be Constant Value Patterns. This feature often helps the trees separate SQLi from benign requests clearly.

How Constant Value Count is dominant:

- Creates very clean splits
- Strong separates SQLi from benign

And due to its high importance, many trees use it early in splitting.

<u>Features of Lower Importance in my model</u>: Double Quote Count, AND Count

- Such features appear in benign traffic and can sometimes be helpful, but they do not consistently nor dramatically reduce impurity. As a result, they get lower average importance.

<u>Limitation</u> – Although feature importance plays a significant role in Random Forest models, it can be biased towards features with higher cardinality or more split opportunities.

Variance in Random Forest

Random Forest reduces the variance of a model.

It’s how sensitive a model is to small changes in the data, a high-variance model will change its decision boundary a lot if you tweak around with your dataset. A low-variance model shouldn’t, it will remain stable.

Single decision trees have high variance because they make greedy splits, and they’re sensitive to small data shifts. If a single tree was used in my case, if a few borderline SQLi samples move sides, the tree may choose a different root split and this would change everything below it.

Bagging

- AKA, bootstrap aggregation, Random Forest uses this to train each tree on a random subset of data. Then it randomly selects features at each split, and averages predictions across many trees.

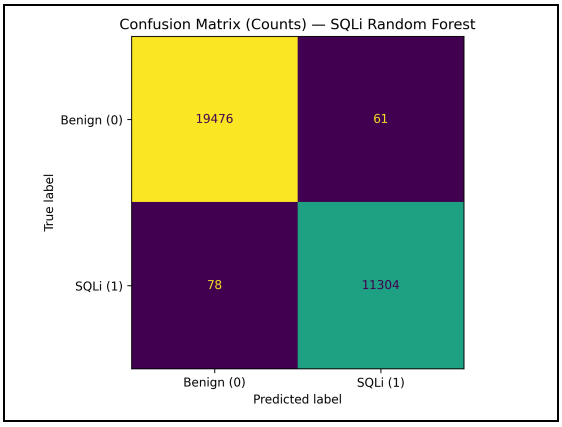
- Mathematically: averaging multiple high-variance models reduces overall variance.

If you look at the trees individually, it may look like overfitting. However, their errors cancel out, and the ensemble becomes stable.

High Variance Model:

- Performs very well on training data
- Performs noticeably worse on unseen data
- Has unstable performance across folds

Confusion Matrix



My model's behaviour based on the confusion matrix:

High recall; sensitive to attacks

Fairly precise, so low false alarms

Balanced and not overly aggressive.

True Negatives, 19476: benign requests correctly classified as benign. So the model is good at not over-flagging normal traffic.

False Positives, 61: these are unnecessary alerts, benign requests incorrectly flagged as SQLi. FPs could:

- Create false alarms
- Block legitimate users
- Overwhelm security analysts

False Negatives, 78: attacks that slipped through as benign, but the rate is only 1.08%, so the model misses very few attacks.

True Positives, 11304: corrected detected, and the recall is high (0.989)

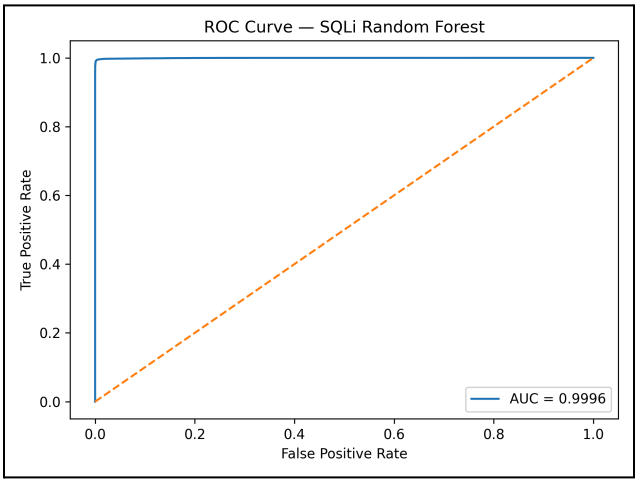
The recall of 0.9931 indicates that over 99.3% of malicious requests are successfully detected, demonstrating high sensitivity to attacks.

The precision of 0.9946 indicates that when the model predicts an attack, it is correct over 99.4% of the time, reflecting low false alarm rates.

Deliverables:

- /outputs/figures/sqli_confusion.png

Receiver Operating Characteristics — ROC Curve



Here, the fixed 0.5 threshold is not used. Changing the decision threshold:

0.9, makes the model strict, resulting in high FN and low FP.

0.2, makes the model aggressive, resulting in low FN and higher FP.

This curve shows how the FP and TP recalls change across all thresholds.

Even when the false positive rate is very small, the true positive rate is already very high. This is why the AUC is 0.999551, the model can catch most attacks.

True Positive Rate (Recall) = TPR = TP / (TP + FN), how many attacks were detected?

False Positive Rate = FPR = FP / (FP + TN), how many benign requests were incorrectly flagged?

Diagonal Line: random guessing, if AUC is approximately 0.5.

My curve:

- High TPR, so most attacks were detected.
- Low FPR, benign requests incorrectly flagged.
- Across many thresholds

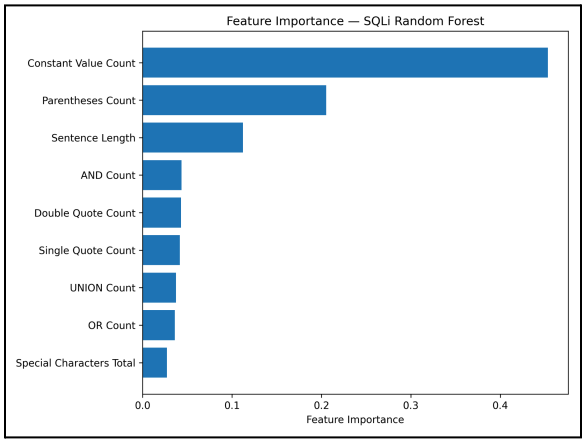
Area Under the ROC Curve (AUC) = 0.995

- Measures ranking quality, it ignores the decision threshold of 0.5.
  0.5 = random guessing
  1.0 = perfect separation
- So if you randomly pick one SQLi and one benign request, the model will rank the SQLi higher 99.5% of the time.

ROC is important in security because the threshold may change in production. ROC will tell us how good the model is at separating classes independent of threshold.

This indicates that the model can effectively rank malicious and benign requests across varying classification thresholds. The high AUC reflects clear probabilistic separation between classes.

Feature Importance



The importance values reflect the average reduction in Gini impurity contributed by each feature across all trees in the Random Forest.

Feature importance is explained in the section above in more depth.

As explained previously, Constant Value Count is the most influential feature in the trained model.

Across all trees and all splits, approximately 45% of the total normalised impurity reduction across all trees is attributed to splits using this feature. This means that across all features, Constant Value Count had the largest contribution to the total impurity reduction.

SQLi frequently uses:

- 1 = 1
- 0 = 0
- TRUE conditions
- Numeric comparisons

And these patterns rarely appear in benign HTTP requests.

Logical Operators:

- These have lower importance, they're useful but only to a certain extent.
- They can appear in benign context but they do not always guarantee maliciousness.
- They only provide moderate impurity reduction, not dominant ones like constant value count.

Limitations:

- Can be biased
- Favors features that create more possible splits
- Feature importance reflects predictive contribution rather than causal influence.

Deliverables:

- /outputs/feature_importance.csv
- /outputs/figures/sqli_feature_importance.png